Exploratory Data Analysis (EDA) and Dashboards

📌 What is Exploratory Data Analysis (EDA)?

> *"The greatest value of a picture is when it forces us to notice what we never expected to see."* — **John Tukey**

Exploratory Data Analysis (EDA) is the process of **summarizing, visualizing, and understanding the structure and patterns in a dataset** — before applying any formal modeling techniques.

Why does EDA matter?

Reason	Detail
🔍 **Understanding Data Structure**	Reveals the organization, dimensions, and relationships within data
🖌️ **Quality Assessment**	Uncovers missing values, outliers, and inconsistencies (EDA and cleaning can be iterative)
💡 **Feature Discovery**	Identifies relevant features and relationships for modeling
📢 **Insight Communication**	Translates raw numbers into clear visual stories for stakeholders

EDA Workflow

`Univariate Analysis → Bivariate Analysis → Multivariate Analysis → Insight Communication`

Each stage builds on the previous one:

- **Univariate**: Understand each variable *alone*
- **Bivariate**: Understand how *two* variables relate
- **Multivariate**: Understand *three or more* variables simultaneously
- **Insight Communication**: Dashboards, reports, visualizations

1. Dataset Overview

🛒 Brazilian E-Commerce (Olist)

> **Why this dataset?**
> It is rich in *every variable type* — continuous prices, categorical regions, temporal timestamps, and ordinal ratings. This mirrors what real e-commerce data scientists work with daily. The **Black Friday 2017 spike** is a memorable teaching moment for pattern discovery.

File	Description
`olist_orders_dataset.csv`	One row per order: `order_id`, `customer_id`, timestamps
`olist_order_items_dataset.csv`	One row per item in an order: `price`, `freight_value`, `seller_id`
`olist_order_reviews_dataset.csv`	Customer reviews: `review_score` (1–5 stars)
`olist_customers_dataset.csv`	Customer info: `customer_id`, `customer_state`

Key Variables

Variable	Type	Description
`price`	Continuous	Item price in Brazilian Real (R$)
`freight_value`	Continuous	Shipping cost in R$
`review_score`	Ordinal (1–5)	Customer satisfaction rating
`order_purchase_timestamp`	DateTime	When the order was placed
`customer_state`	Categorical	Brazilian state abbreviation (e.g., SP, RJ)
`order_delivered_customer_date`	DateTime	When the order arrived
`delivery_days`	Derived Continuous	Calculated: delivered_date − purchase_date

```python
import pandas as pd  # Data manipulation and analysis
import numpy as np  # Numerical computing
import matplotlib.pyplot as plt  # Core plotting (the matplotlib.pyplot module)
import matplotlib.gridspec as gridspec  # Custom grid layouts for dashboards
import seaborn as sns  # Statistical visualizations (built on matplotlib)
from scipy import stats  # Statistical functions (linregress, etc.)
from datetime import timedelta
```

```python
# ── Global Style Settings ──────────────────────────────────────────────
# These settings affect ALL plots in this notebook
# rcParams — short for "runtime configuration parameters",
# a dictionary that stores all default plot settings (colors, font sizes, line widths, DPI, etc.)
plt.rcParams.update(
    {
        'figure.dpi': 120,  # Higher DPI = sharper plots on screen
        'axes.spines.top': False,  # Remove top border from all plots
        'axes.spines.right': False,  # Remove right border from all plots
        'axes.grid': True,  # Show grid lines by default
        'grid.alpha': 0.3,  # Make grid lines semi-transparent
        'grid.linestyle': '--',  # Dashed grid lines
        'font.family': 'sans-serif',  # Clean font
    }
)
```

```
# ── Load the 4 CSV files we need ──────────────────────────────────────
# Make sure all CSV files are in your working directory first.

orders = pd.read_csv('olist_orders_dataset.csv')
order_items = pd.read_csv('olist_order_items_dataset.csv')
reviews = pd.read_csv('olist_order_reviews_dataset.csv')
customers = pd.read_csv('olist_customers_dataset.csv')
```

```
# ── Merge all tables into one working DataFrame ──────────────────────
# We use inner join so we only keep orders that exist in ALL tables.
# 'on' specifies the key column that links the tables.
df = orders.merge(order_items, on='order_id', how='inner')

# Left join for reviews: keep all orders even if review is missing
df = df.merge(reviews[['order_id', 'review_score']], on='order_id', how='left')

# Left join for customer state
df = df.merge(
    customers[['customer_id', 'customer_state']], on='customer_id', how='left'
)
```

```python
# ── Derive: Delivery Time in Days ──────────────────────────────────────
# errors='coerce' converts unparseable strings to NaT (Not a Time) instead of crashing
df['order_delivered_customer_date'] = pd.to_datetime(
    df['order_delivered_customer_date'], errors='coerce'
)
```

```
Columns:
order_id                              str
customer_id                           str
order_status                          str
order_purchase_timestamp        datetime64[us]
order_approved_at                     str
order_delivered_carrier_date          str
order_delivered_customer_date   datetime64[us]
order_estimated_delivery_date         str
order_item_id                       int64
product_id                            str
seller_id                             str
shipping_limit_date                   str
price                             float64
freight_value                     float64
review_score                      float64
customer_state                        str
delivery_days                     float64
```

```python
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])
```

```python
# Subtract datetime columns → gives a timedelta, then .dt.days → integer days
df['delivery_days'] = (
    df['order_delivered_customer_date'] - df['order_purchase_timestamp']
).dt.days
```

```
Shape of merged DataFrame: (113314, 17)
```

```python
# ── Derive: Monthly Order Volume ──────────────────────────────────────
# set_index() makes timestamp the index so .resample() can group by time period
# 'ME' = Month End frequency — counts unique order_ids per calendar month
monthly = (
    df.set_index('order_purchase_timestamp')
    .resample('ME')['order_id']
    .nunique()
    .reset_index()
)
monthly.columns = ['month', 'order_count']  # Rename for clarity
```

```
Shape of monthly DataFrame: (25, 2)

First 3 rows:
      month  order_count
2016-09-30            3
2016-10-31          308
2016-11-30            0

Columns:
month          datetime64[us]
order_count             int64
```

```
monthly = (
    df.set_index('order_purchase_timestamp')
    .resample('ME')['order_id']
    .nunique()
    .reset_index()
)
monthly.columns = ['month', 'order_count']  # Rename for clarity
```

index	order_id	...
2017-01-01	abc123	...
2017-01-15	def456	...

Step 1: `df.set_index('order_purchase_timestamp')` Sets the purchase timestamp column as the DataFrame index. This is required for time-based resampling.

Step 2: `.resample('ME')` Groups the data into monthly buckets. `'ME'` = Month End (last day of each month is used as the label).

Step 3: `['order_id'].nunique()` For each monthly bucket, counts the number of **unique** order IDs. This gives us how many distinct orders were placed each month.

Step 4: `.reset_index()` Converts the index back into a regular column, giving us a clean flat DataFrame.

Step 5: `monthly.columns = ['month', 'order_count']` Renames the columns for clarity.

Each row = one month, with the total number of unique orders placed that month.

Final Output

month	order_count
2017-01-31	452
2017-02-28	701
2017-03-31	889
...	...

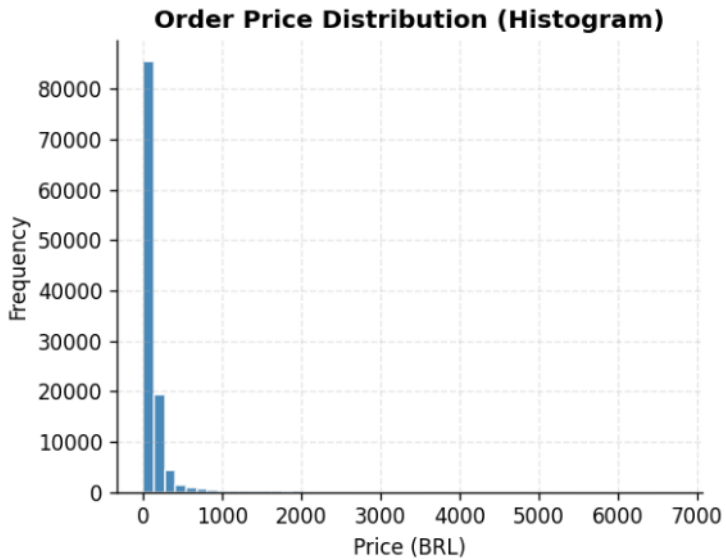
3. Univariate Analysis

Goal of Univariate Analysis: Understand each variable *independently* before looking for relationships. For price, we expect a right skew (most orders are cheap; a few are very expensive). We use three complementary plots to see this from different angles.

Why These Three Together?

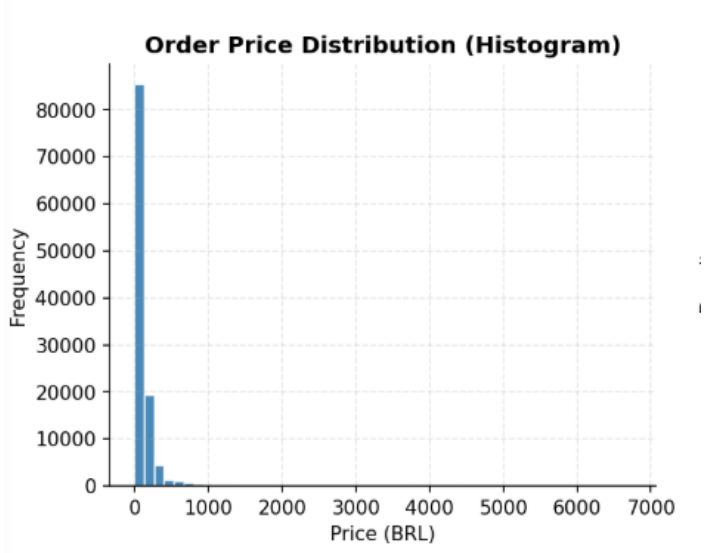
Plot	Strength	Weakness
Histogram	Shows bin-level frequency detail	Sensitive to bin choice
KDE	Reveals smooth shape (multimodality, tails)	Hides exact counts
Box Plot	Immediately shows IQR, median, outliers	Hides shape

Together they give a **complete picture** of the distribution.



What is a Histogram?

A histogram divides a continuous variable into **bins** (intervals) and shows how many data points fall in each bin.



What to look for:

- **Central tendency** — where is the peak (mode)?
- **Spread** — how wide is the distribution?
- **Shape** — symmetric, right-skewed, left-skewed, bimodal?
- **Outliers** — isolated bars far from the main mass?

Key choice: number of bins

Rule	Formula	Best for
Sturges' Rule	$k = 1 + \log_2(n)$	Small datasets
Freedman-Diaconis	$\text{bin\_width} = 2 \times \text{IQR} / n^{(1/3)}$	Large datasets with outliers
Square root	$k = \sqrt{n}$	Quick default

In Matplotlib: `ax.hist(data, bins=50)` — you set `bins` manually.

In Seaborn: `sns.histplot(data, bins=50)` — or use `bins='auto'` to let it decide.

What is a KDE Plot (Kernel Density Estimation)?

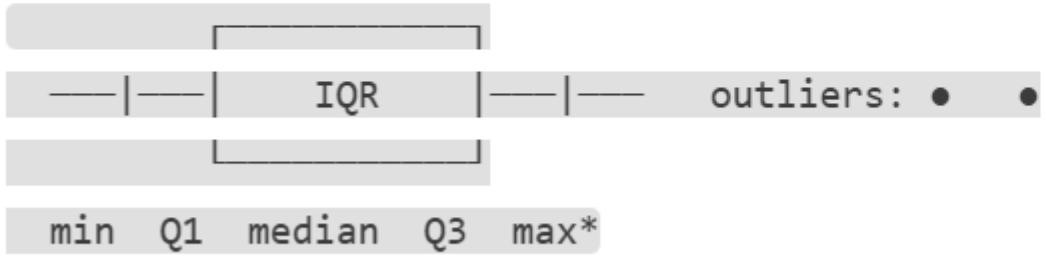
KDE is a non-parametric method for estimating the **probability density function (PDF)** of a continuous variable. Instead of bins, it fits a smooth "kernel" (usually a Gaussian bell curve) at each data point and sums them into one smooth curve.

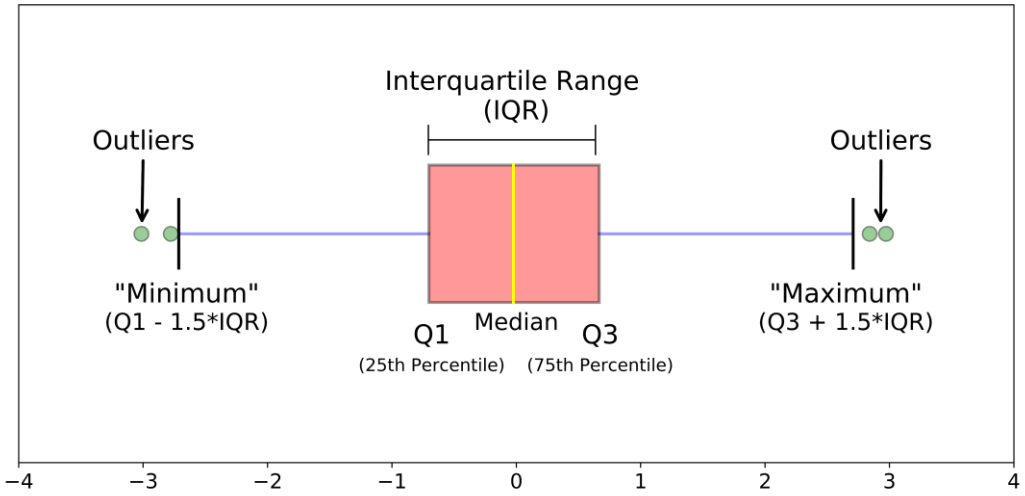
- **Bandwidth** controls smoothness — like bin width in a histogram
- Use `fill=True` to shade the area under the curve
- Great for revealing **multimodality** (multiple peaks) that histograms might miss

In Seaborn: `sns.kdeplot(data=series, fill=True, alpha=0.4)`

What is a Box Plot?

A box plot (box-and-whisker) summarizes 5 key statistics:

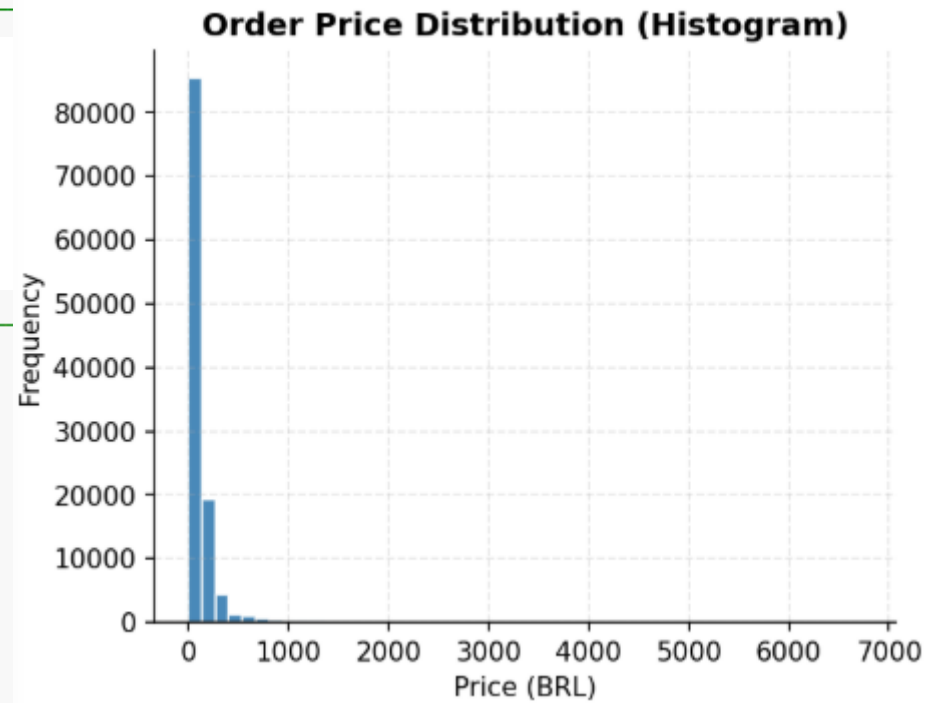




Element	Meaning
Box	Contains the middle 50% of data (IQR = Q3 − Q1)
Line in box	Median (50th percentile)
Whiskers	Extend to min/max within 1.5 × IQR
Dots beyond whiskers	Statistical **outliers**

Box plots are fast to compare groups, but they hide the distribution *shape* (bimodality, skewness). That's why we use KDE alongside them.

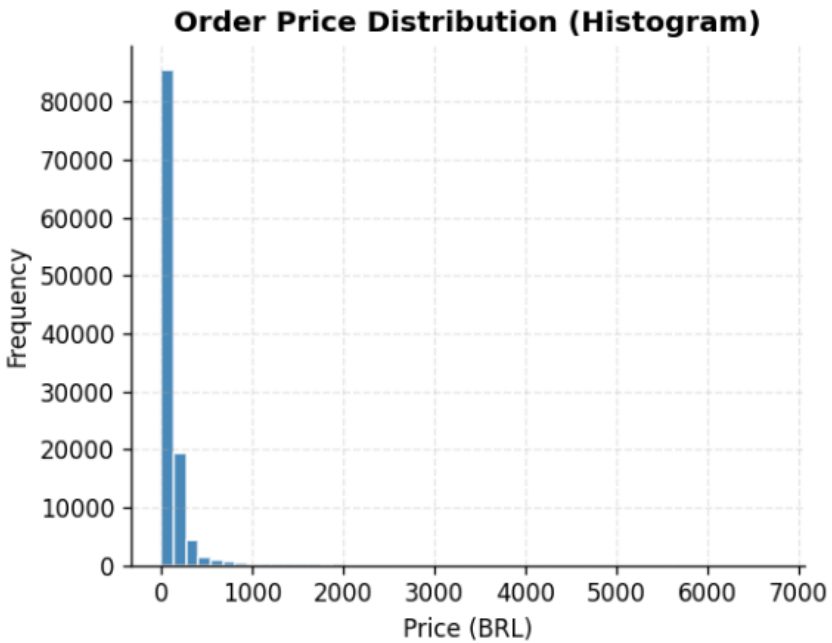
```
# ── 1. UNIVARIATE: Distribution of Order Prices ──────────────────────────

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
# Creates a figure with 1 row and 3 columns of subplots
# axes[0] = histogram, axes[1] = KDE, axes[2] = box plot

# ── Plot 1a: Histogram ──────────────────────────────────────────────────
axes[0].hist(
    df['price'].dropna(),  # Remove NaN values before plotting
    bins=50,  # Number of bins (try 20–100 for continuous data)
    color='#1B6CA8',  # Blue fill color (hex code)
    alpha=0.8,  # 80% opacity — slightly transparent
    edgecolor='white',  # White lines between bars for visual separation
)
axes[0].set_title(
    'Order Price Distribution (Histogram)', fontsize=12, fontweight='bold'
)
axes[0].set_xlabel('Price (BRL)')  # BRL = Brazilian Real
axes[0].set_ylabel('Frequency')  # Y-axis = count of orders in each bin
```



```python
# ── Plot 1b: KDE ──────────────────────────────────────────────────────
sns.kdeplot(
    df['price'].dropna(),  # The data series
    ax=axes[1],  # Which subplot to draw on
    color='#2E8B6E',  # Green color
    fill=True,  # Shade the area under the curve
    alpha=0.4,  # Transparency for the filled area
)
axes[1].set_title('Order Price KDE', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Price (BRL)')
# No ylabel needed — KDE y-axis is "density" (probability per unit)
```


```python
# ── Plot 1c: Box Plot ─────────────────────────────────────────────────
axes[2].boxplot(
    df['price'].dropna(),
    vert=False,  # Horizontal layout (easier to read long tails)
    patch_artist=True,  # Required to fill the box with color
    boxprops=dict(
        facecolor='#EBF4FB', color='#1B6CA8'  # Light blue box fill  # Box border color
    ),
    medianprops=dict(
        color='#D63031',  # Red median line (stands out clearly)
        linewidth=2,  # Thick median line
    ),
)
axes[2].set_title('Order Price Box Plot', fontsize=12, fontweight='bold')
axes[2].set_xlabel('Price (BRL)')
```

```
plt.tight_layout()  # Adjusts spacing between subplots to prevent overlap
plt.savefig('olist_univariate_price.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
count    113314.00
mean        120.48
std         183.28
min           0.85
25%          39.90
50%          74.90
75%         134.90
max        6735.00
Name: price, dtype: float64
```

```
Mean price:   R$ 120.48
Median price: R$ 74.90
Skew indicator: mean > median → RIGHT-skewed
```

`plt.tight_layout()`

Automatically adjusts subplot spacing to prevent titles/labels from overlapping.

Always call this before `plt.show()` when using multiple subplots.

A Nice Side Note: How to understand hexadecimal colors at a glance?

Structure: `#RRGGBB`

- First 2 digits → **Red**
- Middle 2 digits → **Green**
- Last 2 digits → **Blue**

Each channel ranges from `00` (none) to `ff` (full).

Key values to memorize:

- `00` = 0% of that color
- `80` = ~50% of that color
- `ff` = 100% of that color

Quick rules:

- **All 3 equal** (`#404040`, `#abababa`) → always a shade of **gray**
- **Darker** = digits closer to `00`
- **Lighter** = digits closer to `ff`
- **Pastel** = high values on all channels (e.g. `#ffccdd`)
- **Neon/vivid** = one channel at `ff`, others low

Common patterns:

Hex	Logic	Color
`#ff0000`	full red, no green, no blue	🔴 Red
`#00ff00`	no red, full green, no blue	🟢 Green
`#0000ff`	no red, no green, full blue	🔵 Blue
`#ffff00`	full red + green, no blue	🟡 Yellow
`#ff00ff`	full red + blue, no green	🟣 Magenta
`#00ffff`	full green + blue, no red	💙 Cyan
`#ffffff`	all full	⬜ White
`#000000`	all zero	⬛ Black
`#808080`	all ~50%	🩶 Gray

4. Bivariate Analysis

Goal of Bivariate Analysis: Understand how two variables relate to each other. We explore a continuous–continuous relationship (price vs freight) and a categorical–continuous relationship (review score vs price).

What is a Scatter Plot + Regression Line?

A **scatter plot** places each data point as a dot at coordinates `(x, y)`, showing the raw relationship between two continuous variables.

Adding a regression line (also called a trend line or best-fit line):

- Summarizes the overall direction: **positive** (both rise together), **negative** (one rises as the other falls), or **flat** (no relationship)
- The **Pearson correlation coefficient r** quantifies this:

r value	Interpretation
`r ≈ +1.0`	Perfect positive linear relationship
`r ≈ +0.3 to 0.7`	Moderate positive relationship
`r ≈ 0`	No linear relationship
`r ≈ -0.3 to -0.7`	Moderate negative relationship
`r ≈ -1.0`	Perfect negative linear relationship

We use `scipy.stats.linregress(x, y)` which returns: slope `m`, intercept `b`, correlation `r`, p-value, and standard error.

What is a Box Plot by Category?

When comparing a **continuous variable** (price) across **groups** (review scores 1–5), grouped box plots are the right choice:

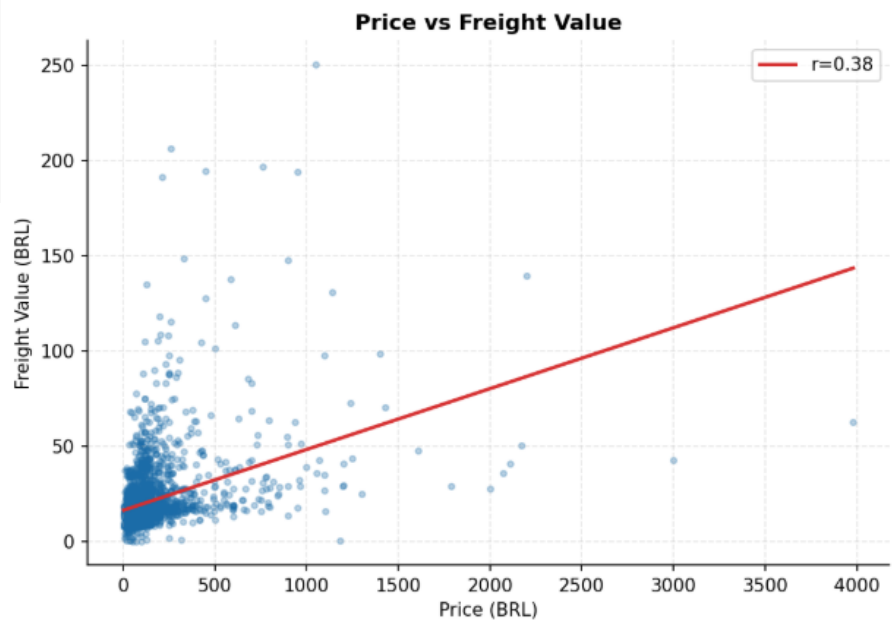
- Each group gets its own box, showing median and spread
- You can immediately see: *"Do 5-star orders have different price distributions than 1-star orders?"*
- The **ordinal** nature of review scores (1 < 2 < 3 < 4 < 5) is preserved by the x-axis ordering

```python
# ── 2. BIVARIATE: Price vs Freight + Review by Category ──────────────
# WHY: Scatter reveals correlation between order value and shipping cost.
#      Box plots compare distributions across groups.
```

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```python
# ── Plot 2a: Scatter + Regression Line (Price vs Freight) ─────────────
# Sample 3000 rows to avoid overplotting (100k points would be a solid blob)
sample = df[['price', 'freight_value']].dropna().sample(3000, random_state=42)
```

```python
axes[0].scatter(
    sample['price'],  # X-axis: order price
    sample['freight_value'],  # Y-axis: shipping cost
    alpha=0.3,  # Low opacity → overlapping points show density
    color='#1B6CA8',
    s=10,  # s = marker size in points² (small → less clutter)
)
```



```python
# Compute regression: returns slope m, intercept b, correlation r, p-value, std_err
m, b, r, p, _ = stats.linregress(sample['price'], sample['freight_value'])
# m = slope (how much freight increases per R$1 increase in price)
# b = y-intercept (freight cost when price = 0)
# r = Pearson correlation coefficient (-1 to +1)
# p = p-value (< 0.05 means statistically significant)

# Draw the regression line: y = m*x + b
x_line = np.linspace(sample['price'].min(), sample['price'].max(), 100)
axes[0].plot(
    x_line,
    m * x_line + b,
    color='#D63031',
    lw=2,  # Red line, linewidth=2
    label=f'r={r:.2f}',
)  # Show r value (correlation) in legend

axes[0].legend()  # Display the label we set above
axes[0].set_xlabel('Price (BRL)')
axes[0].set_ylabel('Freight Value (BRL)')
axes[0].set_title('Price vs Freight Value', fontsize=12, fontweight='bold')
```

```python
# ── Plot 2b: Box Plots by Review Score ──────────────────────────────────
# Build a list: one array of prices per each review score (1, 2, 3, 4, 5)
review_data = [df[df['review_score'] == s]['price'].dropna() for s in range(1, 6)]
# list of 5 pd Series
# review_data[0] = all prices where review_score == 1
# review_data[4] = all prices where review_score == 5

bp = axes[1].boxplot(
    review_data,  # list of 5 pd Series
    patch_artist=True,  # Fill boxes with color
    labels=[1, 2, 3, 4, 5],  # X-axis tick labels
)

# Apply a color gradient on review groups: red (bad review) → yellow → green (good review)
colors = ['#D63031', '#E17055', '#FDCB6E', '#74B9FF', '#00B894']
for patch, c in zip(bp['boxes'], colors):
    patch.set_facecolor(c)  # Color each box differently

axes[1].set_title('Order Price by Review Score', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Review Score')
axes[1].set_ylabel('Price (BRL)')
```

How does `plt.boxplot()` know whether to draw one box or many?

It depends on **what data you pass in** — specifically how many array-like objects (lists, Series, arrays) you provide.

One boxplot — pass a single array-like:

```
plt.boxplot([10, 20, 15, 30, 25])
```

One array → one box.

Multiple boxplots — pass a list of array-likes:

```
plt.boxplot([
    [10, 20, 15],          # group 1 — plain list
    pd.Series([30, 25]),   # group 2 — Series
    np.array([50, 45, 55]) # group 3 — numpy array
])
```

Each array-like → one box. Three of them → three boxes side by side.

In your case (grouped by review score):

```
groups = [df[df['review_score'] == score]['price'] for score in [1,2,3,4,5]]
plt.boxplot(groups)
```

You're passing 5 Series (one per score) → matplotlib draws 5 boxes automatically.

```
plt.tight_layout()
plt.savefig('olist_bivariate.png', dpi=150, bbox_inches='tight')
plt.show()
```

Regression equation: Freight = 0.032 × Price + 16.18

Pearson r = 0.384 → Weak positive correlation

Plot 1 — Price vs Freight Value (Scatter)

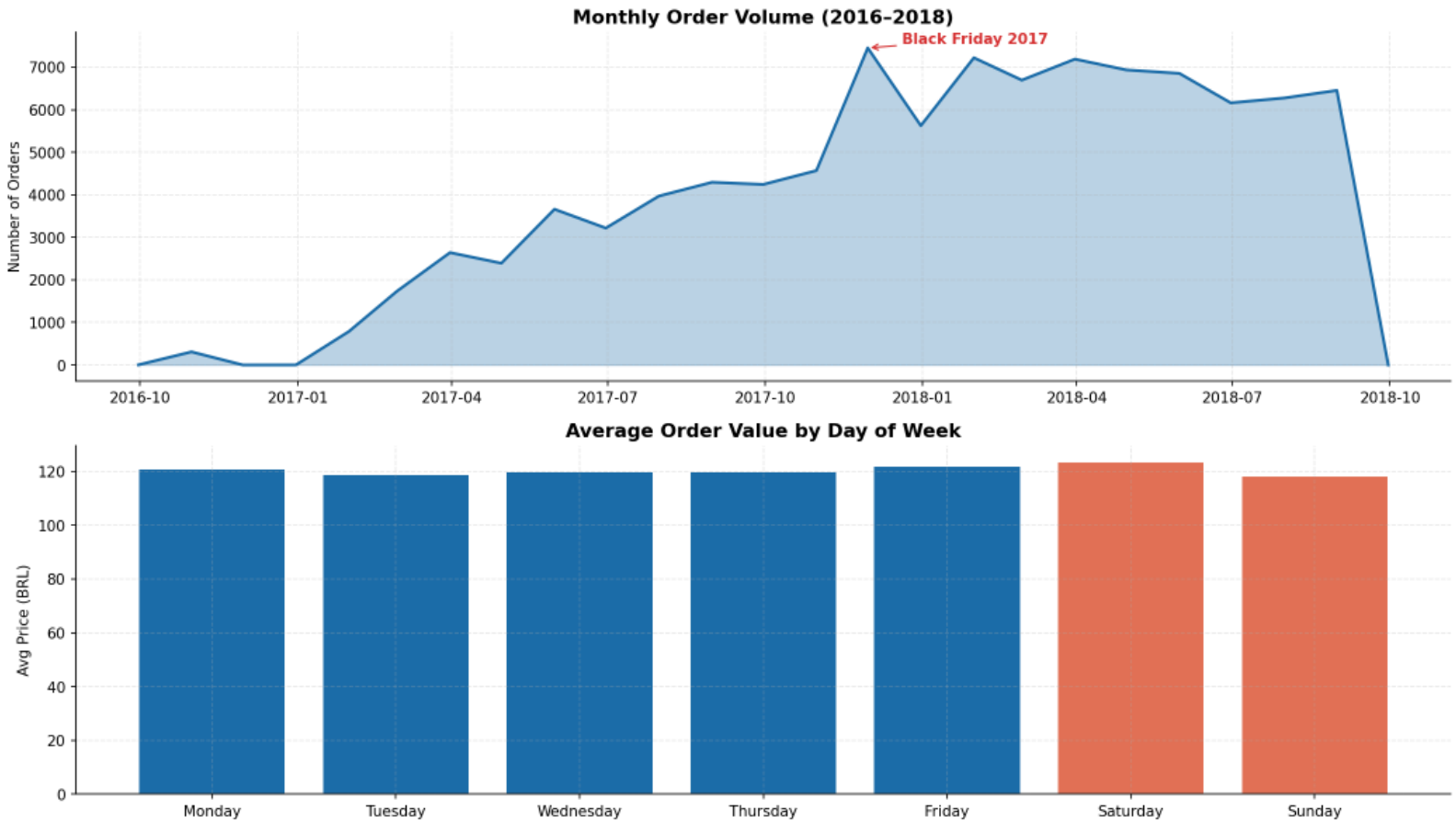
- **Weak positive correlation (r = 0.38)** — as price increases, shipping cost tends to rise, but loosely
- Most orders cluster at **low price (0–500 BRL) and low freight (0–50 BRL)** — the bulk of business is cheap, light items
- **High variance at low prices** — freight can be anywhere from 0 to 200 BRL even for cheap items, suggesting size/weight matters more than price for shipping
- A few **extreme outliers** — items priced at 3000–4000 BRL but with relatively low freight, possibly digital or small luxury goods

Plot 2 — Order Price by Review Score (Boxplot)

- **Medians are nearly identical across all scores (1–5)** — price alone doesn't explain customer satisfaction
- **All boxes are extremely compressed near zero** with massive outliers above — meaning most orders are low-priced regardless of review score
- **Score 4 and 5 have the most extreme outliers** (up to ~4800 BRL) — high-value purchases still tend to get good reviews
- **Score 1 has notable high-value outliers too** — expensive items receiving bad reviews could indicate unmet expectations

5. Pattern Discovery: Time-Series & Seasonality

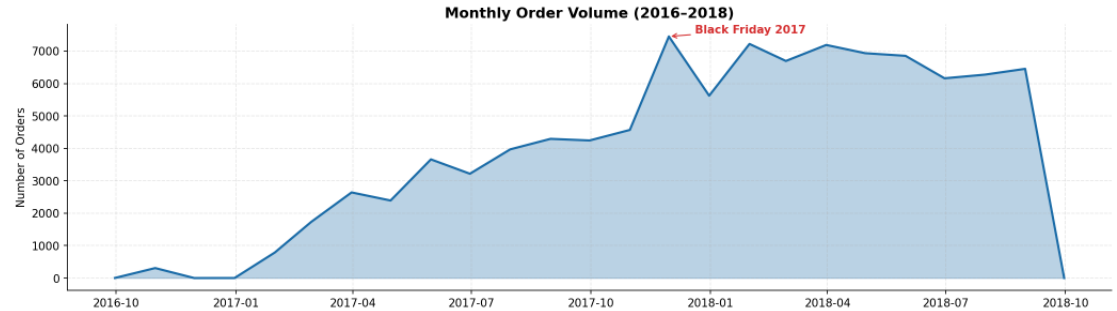
Principle: With temporal data, always plot the time series first before any aggregation. Annotated line charts communicate findings to stakeholders effectively.



What is a Time-Series Line Chart?

A **time-series chart** plots values over time along the x-axis. For business data:

- **Trend** = long-term direction (growth, decline, flat)
- **Seasonality** = recurring patterns (weekly, monthly, yearly)
- **Anomalies** = unexpected spikes or drops (Black Friday, COVID, etc.)



Key techniques:

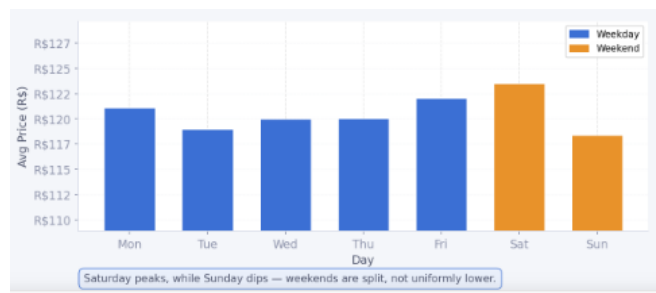
- `fill_between()` fills the area under the line → visually emphasizes volume
- `annotate()` adds arrows pointing to key events → helps stakeholders understand *why* there's a spike

---

What is a Day-of-Week Bar Chart (Seasonality)?

By grouping orders by **day of week** and computing the average order value, we can detect **weekly seasonality**:

- Are Monday orders more valuable than Sunday orders?
- Does purchase behavior follow work-week patterns?

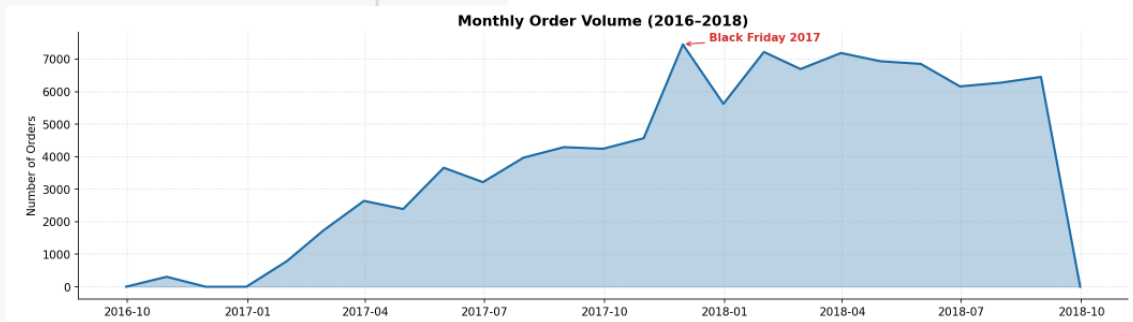


We use `df['timestamp'].dt.day_name()` to extract the day name from a datetime column.

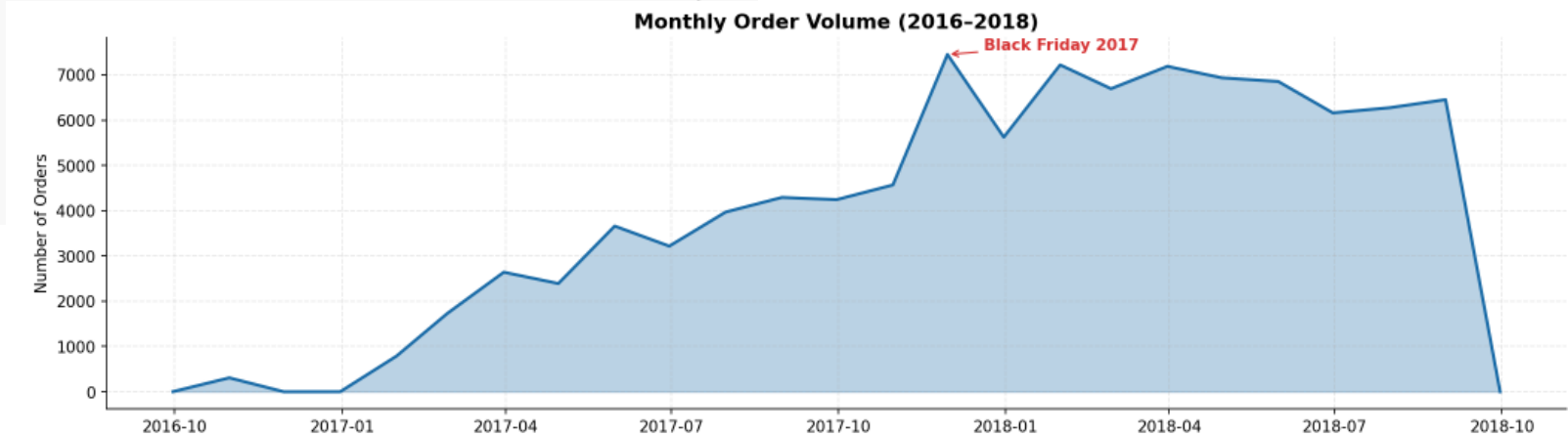
```
# ── 3. PATTERN DISCOVERY: Monthly Orders + Seasonality ──────────────────
# WHY: Time-series reveals the Nov 2017 Black Friday spike
#      and overall growth trend --- critical business insight.
```

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8))
# 2 rows, 1 column → top chart for monthly trend, bottom for day-of-week
```

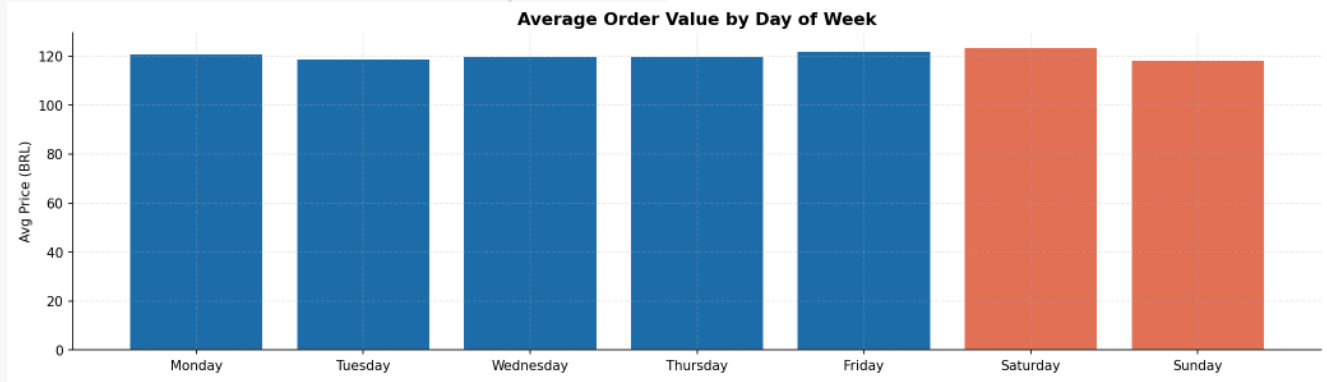
```
# ── Plot 3a: Monthly Order Volume Trend ─────────────────────────────────
# fill_between: shades the area between y=0 and the data line
axes[0].fill_between(
    monthly['month'],  # X-axis: datetime values
    monthly['order_count'],  # Y-axis: number of orders
    alpha=0.3,  # Light fill (mostly see the line)
    color='#1B6CA8',
)
# Plot the line on top of the fill
axes[0].plot(
    monthly['month'],
    monthly['order_count'],
    color='#1B6CA8',
    linewidth=2,  # Thicker line for the trend
)
axes[0].set_title('Monthly Order Volume (2016–2018)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Number of Orders')
```



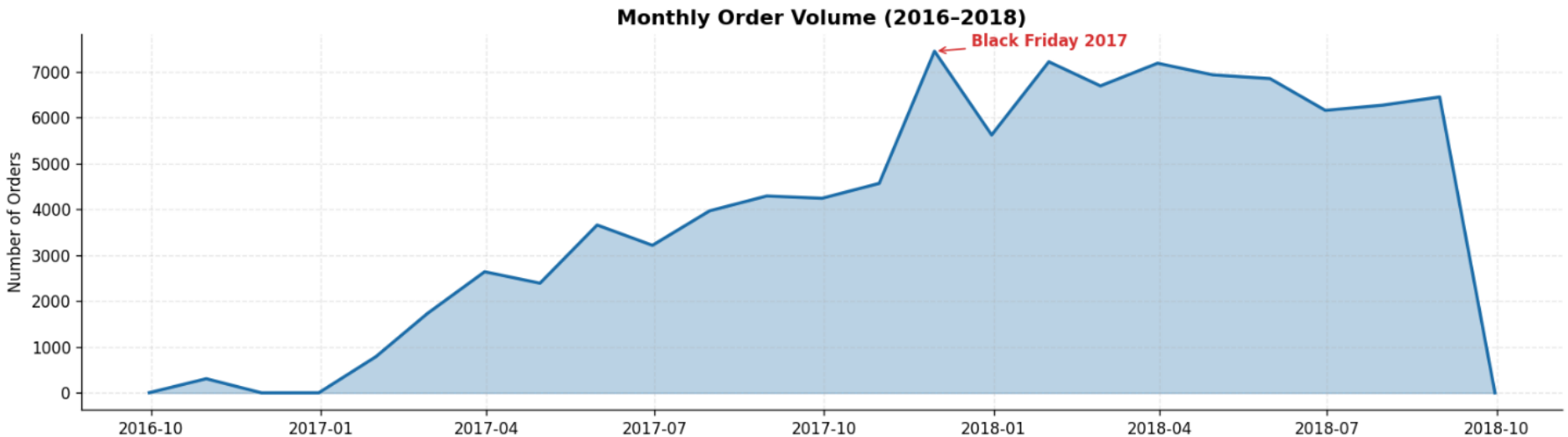
```python
# ── Annotate Black Friday 2017 ──────────────────────────────────────
# Find the month with the maximum order count
# idxmax(): return the row label of the maximum value.
peak_month = monthly.loc[monthly['order_count'].idxmax(), 'month']
peak_val = monthly['order_count'].max()

axes[0].annotate(
    'Black Friday 2017',  # The text to display
    xy=(peak_month, peak_val),  # Arrow tip: points TO this location
    xytext=(
        peak_month + timedelta(days=20),
        peak_val + 100,
    ),  # Text location:  ABOVE the peak (+100) and shifted to RIGHT (by 10 days)
    arrowprops=dict(
        arrowstyle='->',  # Arrow style: '->' = simple arrow
        color='#D63031',  # Red arrow
    ),
    color='#D63031',  # Red text
    fontweight='bold',
)
```

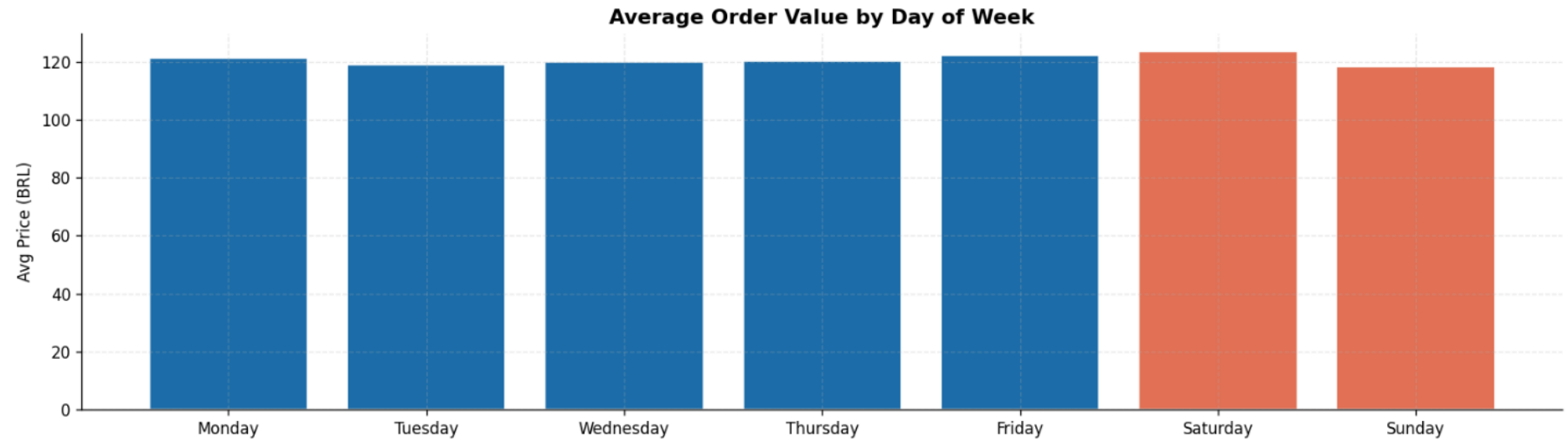


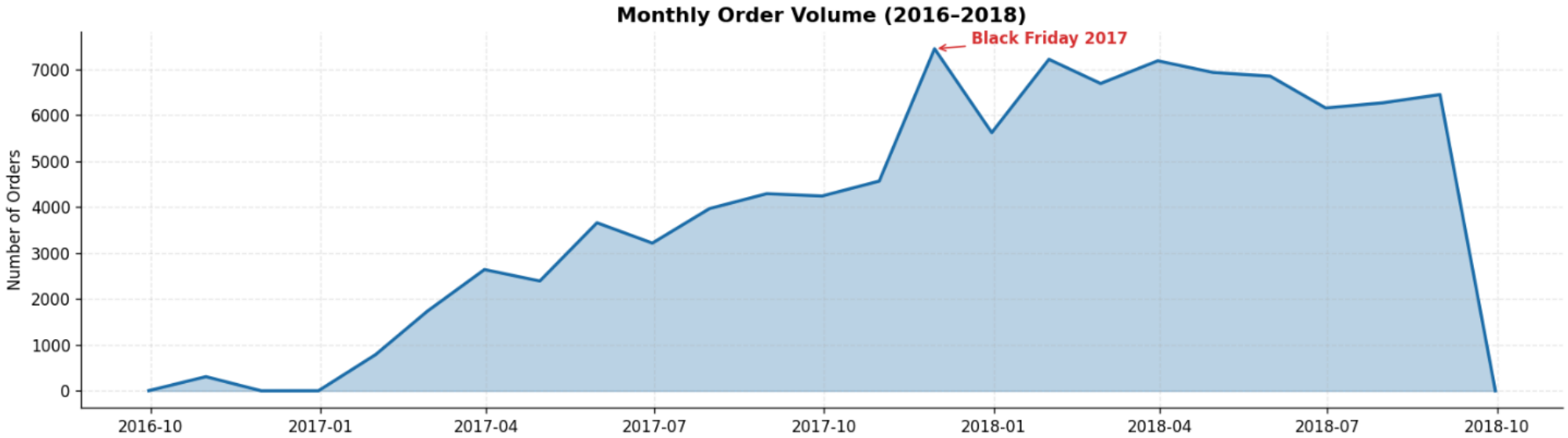
```python
# ── Plot 3b: Day-of-Week Seasonality ──────────────────────────────────────
# Extract day name from the datetime column
# dow: day of week
df['dow'] = df['order_purchase_timestamp'].dt.day_name()
# dt.day_name() → 'Monday', 'Tuesday', ... 'Sunday'

# Reindex to enforce correct Mon→Sun order (groupby sorts alphabetically otherwise)
dow_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday',
]
dow_avg = df.groupby('dow')['price'].mean().reindex(dow_order)
# groupby('dow') → groups rows by day name
# ['price'].mean() → average price per group
# .reindex(dow_order) → reorders from Mon→Sun

# Color weekdays blue, weekends orange
colors = ['#1B6CA8'] * 5 + ['#E17055'] * 2

axes[1].bar(
    dow_avg.index,  # X-axis: day names
    dow_avg.values,  # Y-axis: average order value
    color=colors,
    edgecolor='white',
)
axes[1].set_title('Average Order Value by Day of Week', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Avg Price (BRL)')
```



```
plt.tight_layout()
plt.savefig('olist_timeseries.png', dpi=150, bbox_inches='tight')
plt.show()
```





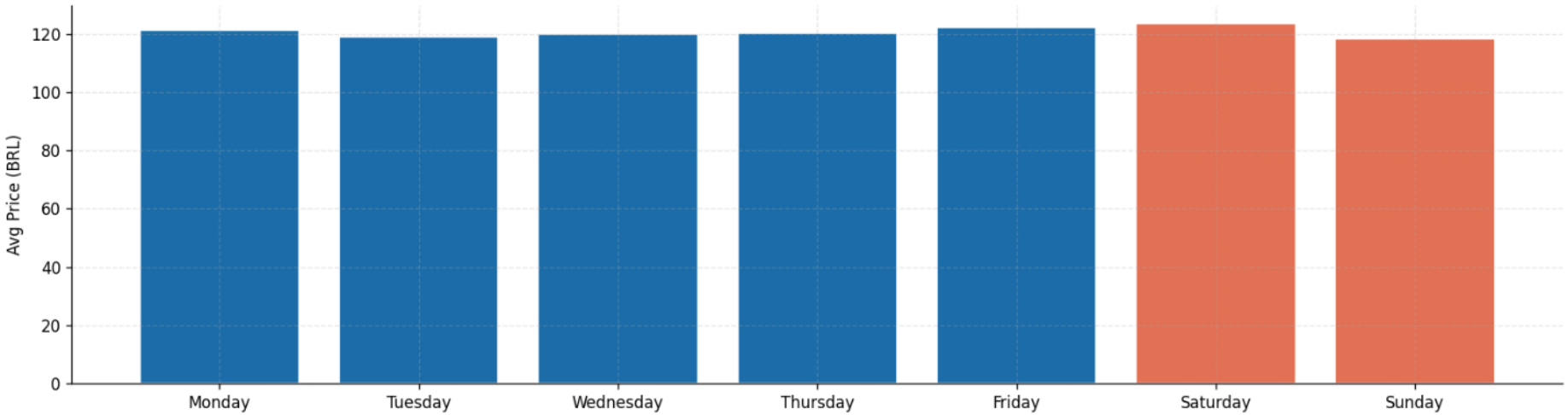


Plot 1 — Monthly Order Volume (2016–2018)

- **Strong overall growth** — orders grew from near zero in late 2016 to ~7000/month by early 2018, suggesting the business was scaling rapidly
- **Black Friday 2017 spike** — a sharp peak in Nov 2017 (~7500 orders), the highest single month, driven by seasonal shopping
- **Dip after Black Friday** — Dec 2017 dropped noticeably, likely a post-event slowdown, then recovered
- **Plateau in 2018** — growth flattened between 6000–7000 orders from Jan–Aug 2018, suggesting market saturation or slower acquisition
- **Sharp cliff at Oct 2018** — orders drop to near zero, almost certainly **incomplete data** for that month, not a real business collapse

Plot 2 — Average Order Value by Day of Week

- **Remarkably flat across all days** — all bars sit between ~118–124 BRL, meaning customers spend roughly the same amount regardless of the day
- **Weekends (Saturday, Sunday) are slightly higher** — Saturday peaks at ~123 BRL, suggesting weekend shoppers may buy slightly more premium items
- **No strong weekday effect** — unlike many e-commerce platforms, there's no "Monday motivation" or "Friday splurge" pattern here



Key takeaway:

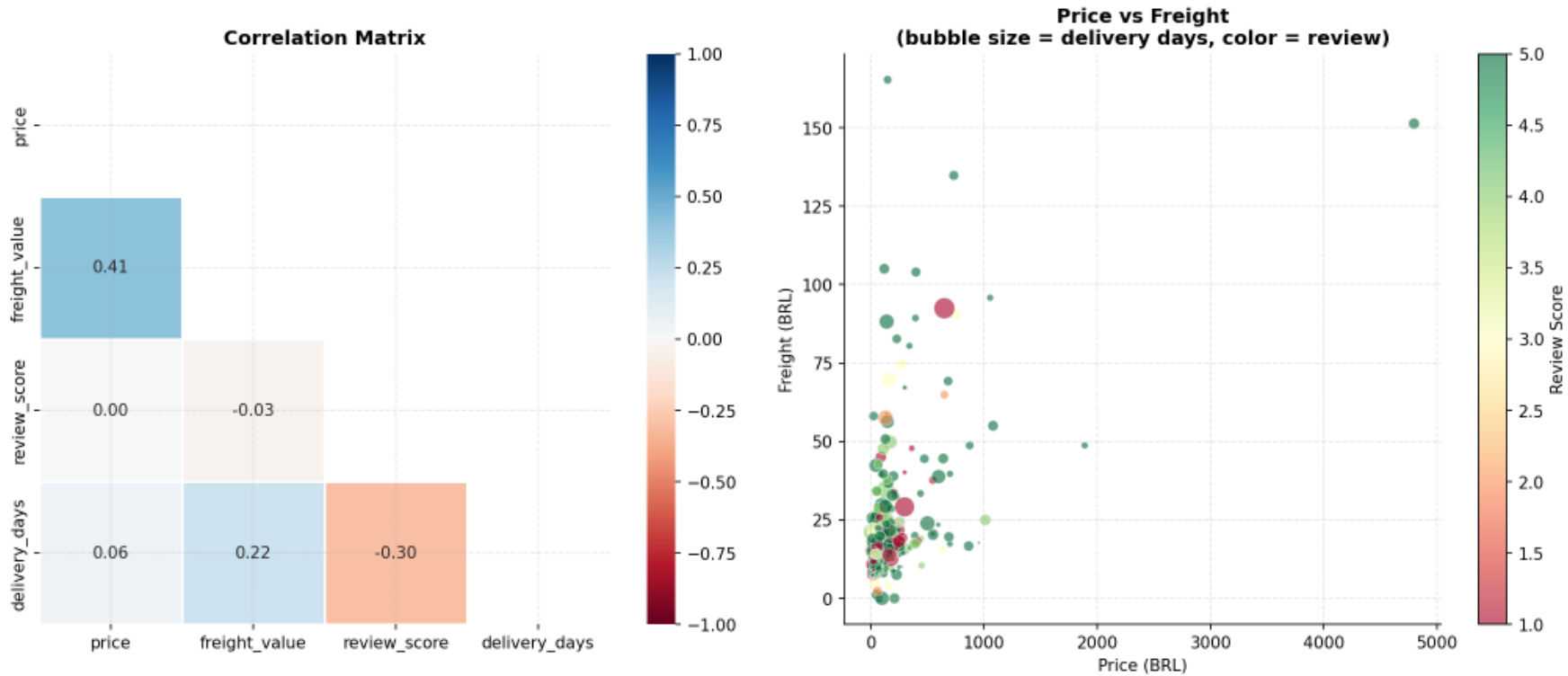
> The business grew aggressively through 2017, peaked on Black Friday, then stabilized — but customer spending behavior is highly consistent regardless of the day, meaning promotions targeting specific days would likely have limited impact.

6. Multivariate Analysis

> **Goal:** Study three or more variables simultaneously to capture complex interdependencies. We use a correlation heatmap (all pairwise) and a bubble chart (4 variables in one plot).

```
# ── 4. MULTIVARIATE: Correlation Heatmap + Bubble Chart ──────────────────
# WHY: Correlation matrix flags multicollinearity before modeling.
#      Bubble chart encodes 4 variables simultaneously.
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```



What is a Correlation Heatmap?

A **correlation matrix** shows the **Pearson correlation coefficient (r)** between every pair of numeric variables.
Each cell contains a value from −1 to +1:

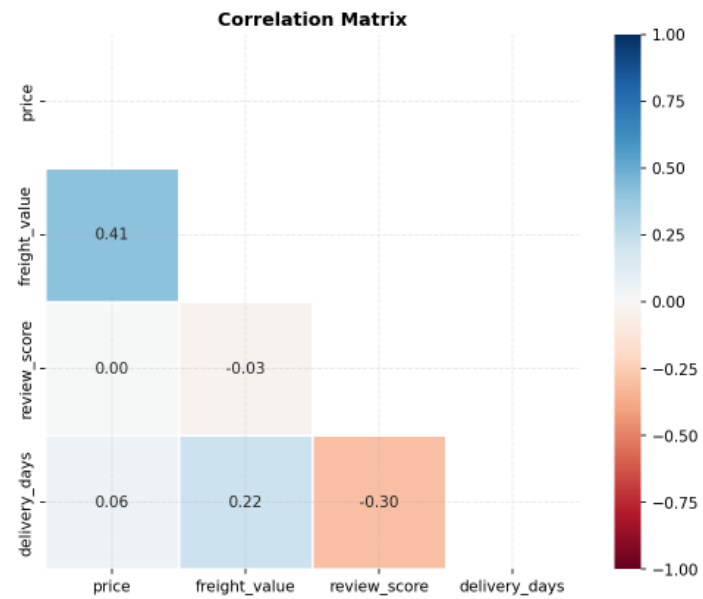
- **r = +1** → perfect positive linear relationship (as x grows, y grows proportionally)
- **r = 0** → no linear relationship
- **r = −1** → perfect negative linear relationship (as x grows, y shrinks proportionally)

Why mask the upper triangle?

A correlation matrix is symmetric: `corr(A, B) == corr(B, A)`. Showing both triangles is redundant.
`np.triu()` creates a mask for the upper triangle so we only show the lower half — cleaner and easier to read.

Diverging colormap: We use `'RdBu_r'` (Red-Blue reversed):

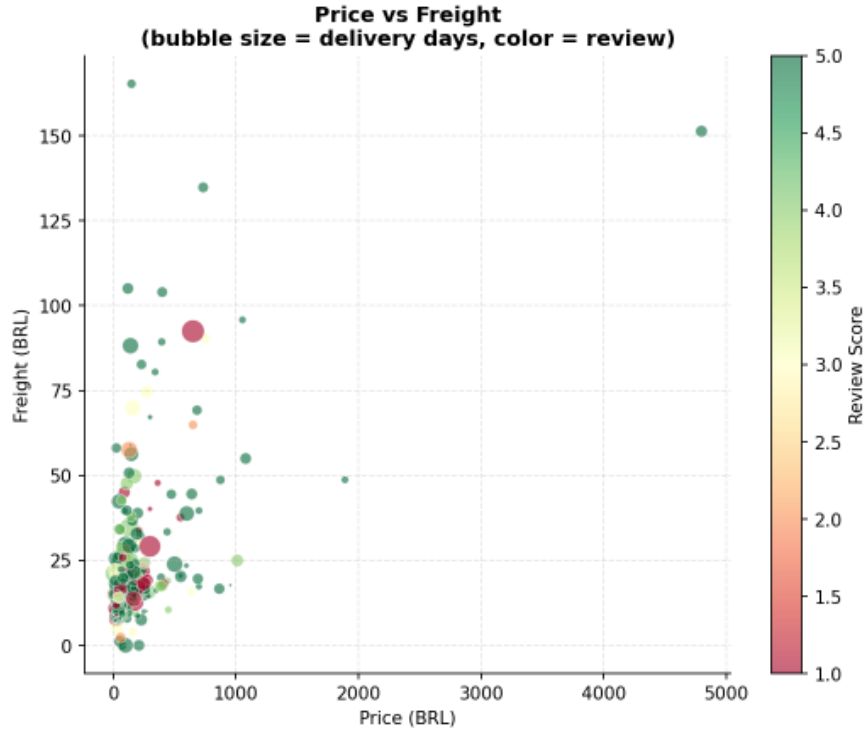
- **Deep blue** → strong negative correlation (e.g., fast delivery → high review score)
- **White** → no correlation (r ≈ 0)
- **Deep red** → strong positive correlation



What is a Bubble Chart?

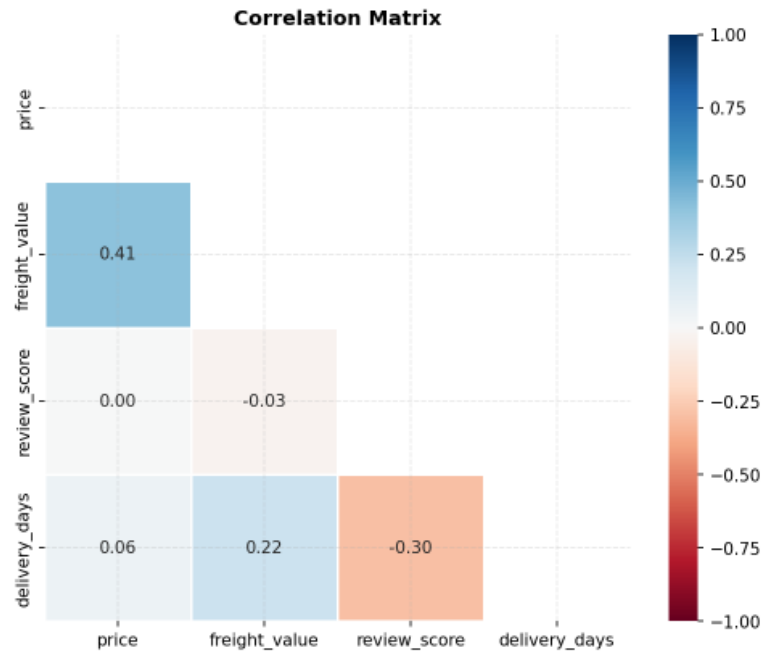
A bubble chart extends a scatter plot to encode **4 variables simultaneously**:

Dimension	Variable
X-axis	price
Y-axis	freight_value
Bubble size (s)	delivery_days (larger bubble = slower delivery)
Bubble color (c)	review_score (red = bad, green = good)



This is powerful for identifying *patterns across multiple dimensions at once* — something impossible with 2D plots alone.

```python
# ── Plot 4a: Correlation Heatmap ──────────────────────────────────────
num_cols = ['price', 'freight_value', 'review_score', 'delivery_days']
corr = df[num_cols].dropna().corr()
# .corr() computes the Pearson correlation matrix — result is a 4x4 DataFrame
# Each cell [i,j] = Pearson r between column i and column j

# Create a mask for the upper triangle (True = masked/hidden)
mask = np.triu(np.ones_like(corr, dtype=bool))
# np.ones_like(corr) → 4x4 matrix of ones (same shape as corr)
# np.triu() → keeps upper triangle as True, lower as False
# dtype=bool → convert to boolean (True/False)

sns.heatmap(
    corr,
    ax=axes[0],
    annot=True,  # Show the r value in each cell
    fmt='.2f',  # Format: 2 decimal places (e.g., 0.39)
    mask=mask,  # Hide the upper triangle (it's redundant) --> True means mask out
    cmap='RdBu',  # Red-Blue reversed: red=positive, blue=negative
    center=0,  # 0 correlation = white (neutral)
    vmin=-1,
    vmax=1,  # Fix scale from -1 to +1
    square=True,  # Force square cells
    linewidths=0.5,  # Thin lines between cells for visual separation
)
axes[0].set_title('Correlation Matrix', fontsize=12, fontweight='bold')
```



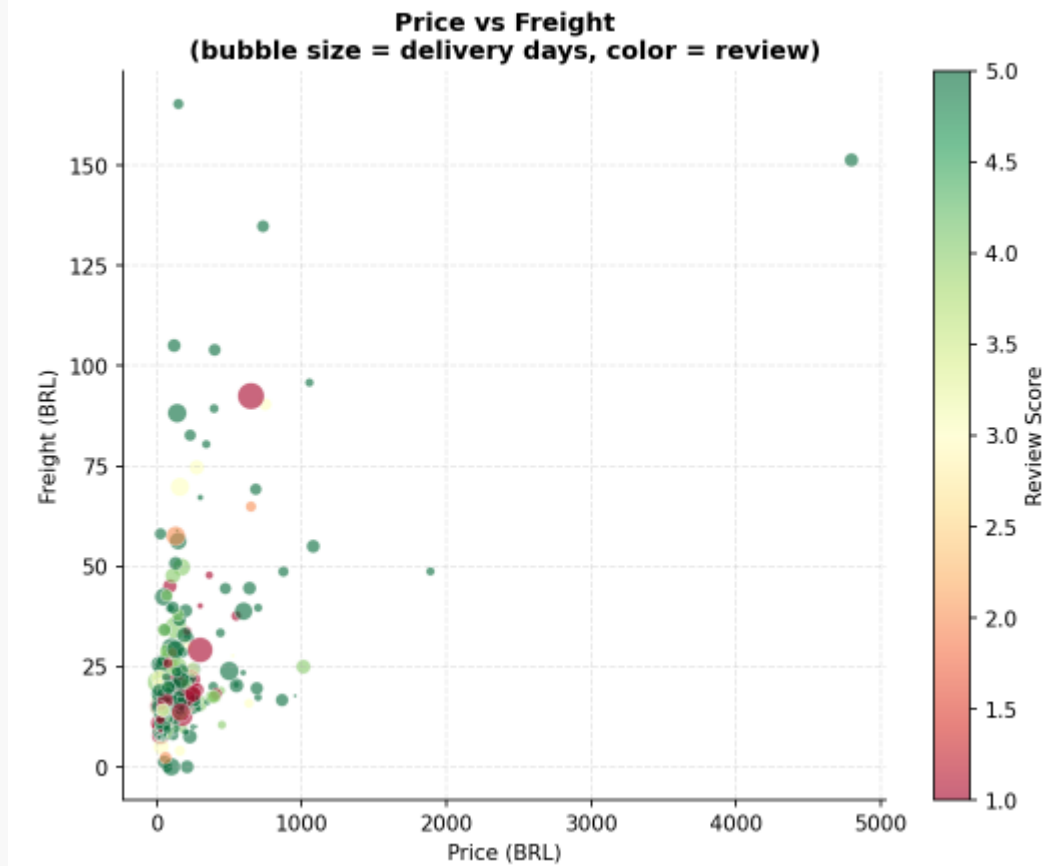
`cmap='RdBu_r'` — the colormap. Controls what colors map to what values.

Common options for correlation heatmaps:

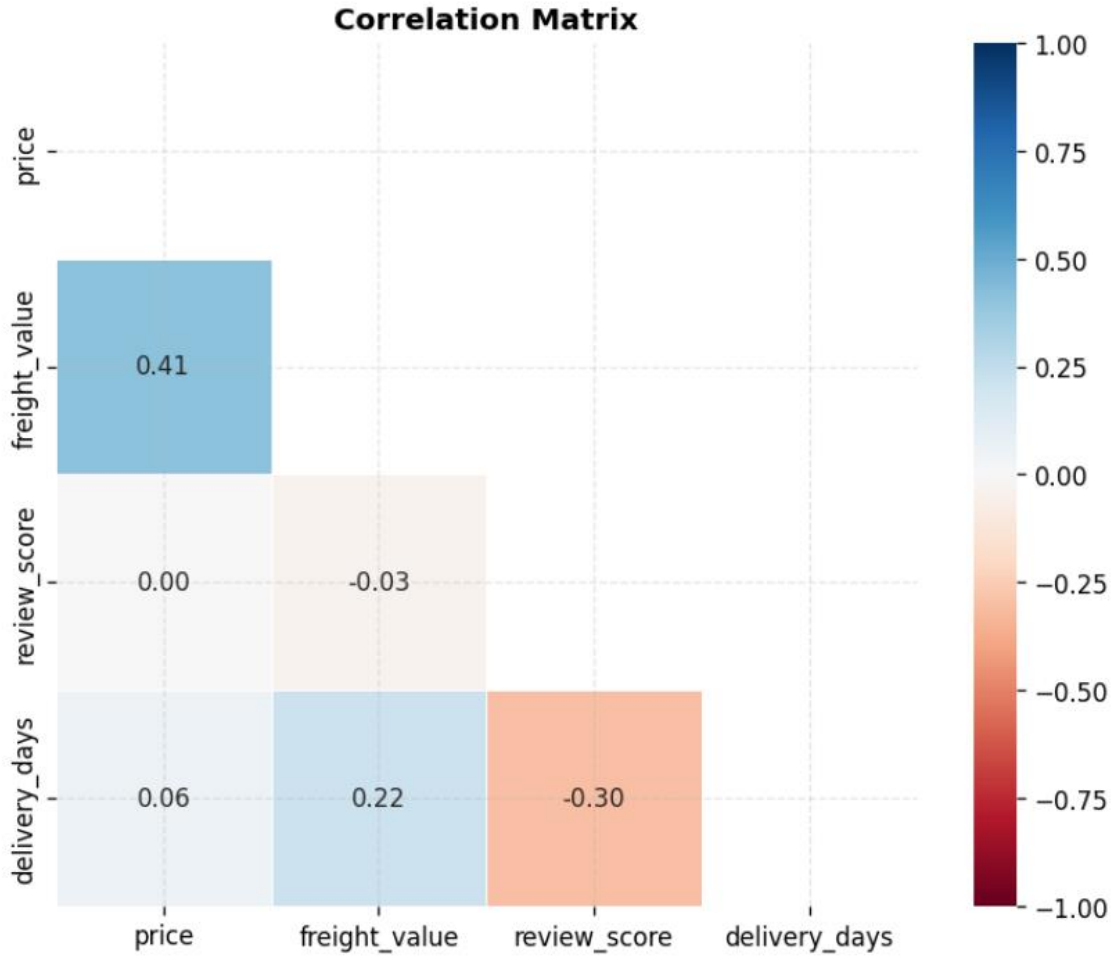
cmap	Colors	Good for
`'RdBu'`	blue=positive, red=negative	correlation (default direction)
`'RdBu_r'`	red=positive, blue=negative	correlation (reversed — more intuitive)
`'RdYlGn'`	red → yellow → green	performance, scores
`'RdYlGn_r'`	green → yellow → red	risk, errors
`'coolwarm'`	blue → white → red	similar to RdBu_r, softer
`'PiYG'`	pink → light yellow like white → green	diverging, colorblind-friendly
`'seismic'`	blue → white → red	high contrast diverging

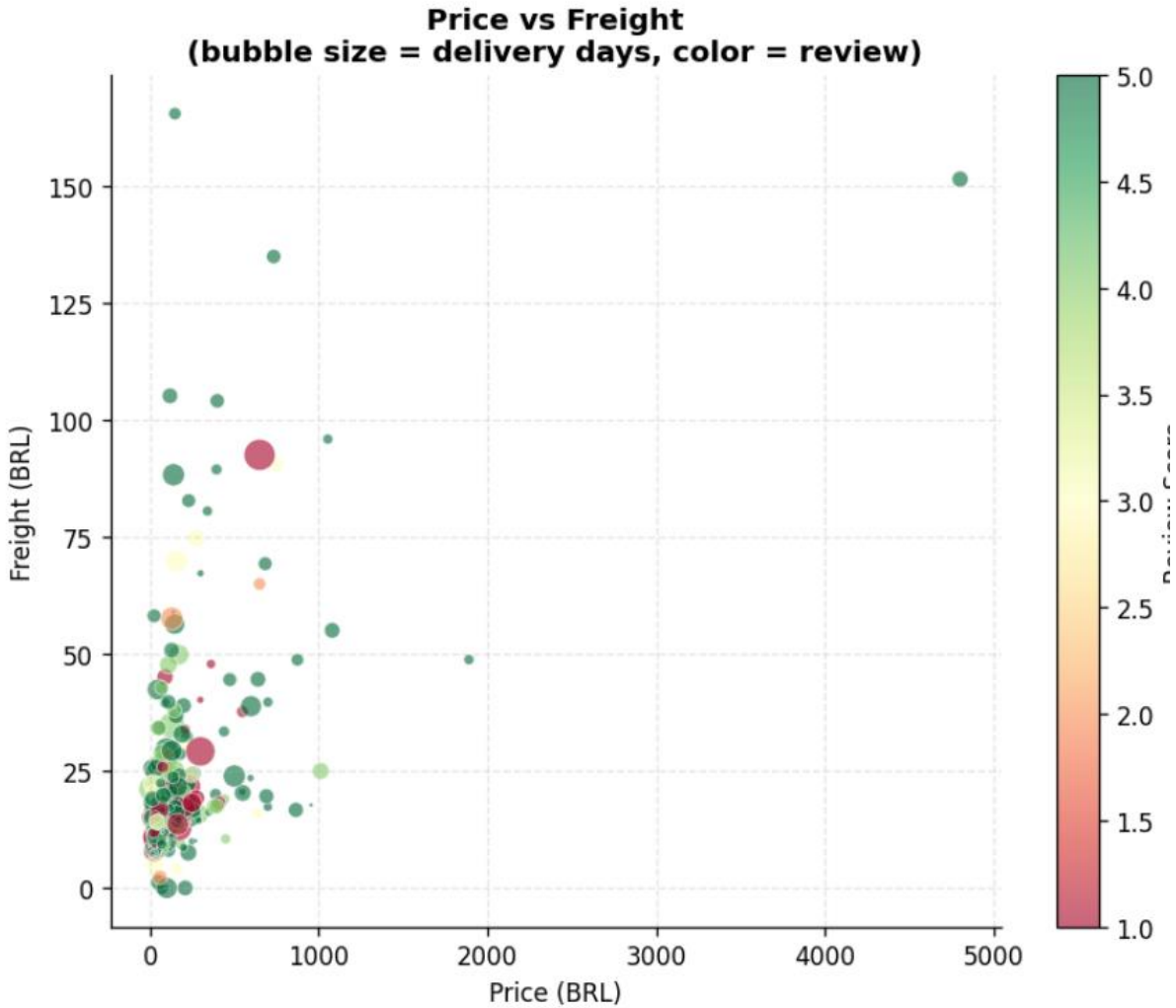
What does `_r` (reversed) mean? Every matplotlib colormap has a reversed version — just append `_r` to flip the color direction end-to-end.

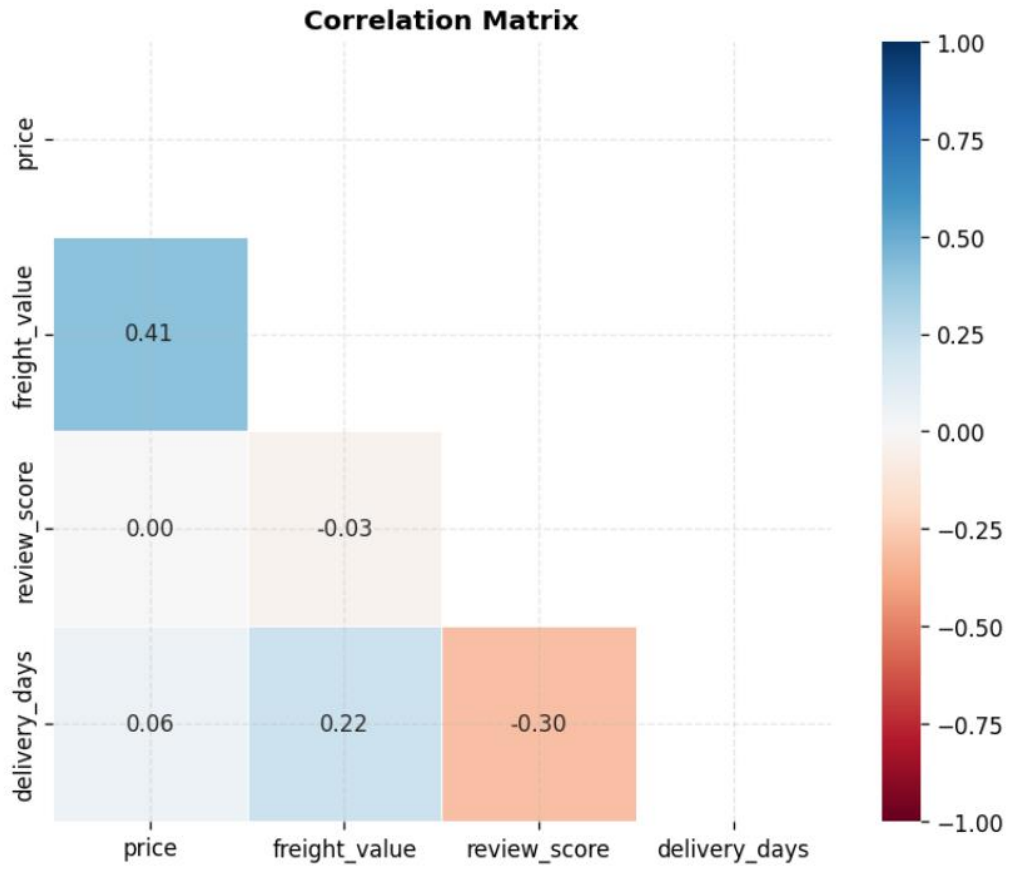
```python
# ── Plot 4b: Bubble Chart ──────────────────────────────────────────
# Sample 500 points (scatter with all 100k would be unreadable)
sample2 = df[num_cols].dropna().sample(500, random_state=42)

scatter = axes[1].scatter(
    sample2['price'],  # X-axis: order price
    sample2['freight_value'],  # Y-axis: freight cost
    s=sample2['delivery_days'].clip(1, 60) * 3,
    # s = bubble size: delivery_days × 3 (scale up for visibility)
    # .clip(1, 60) caps extreme values (prevents enormous bubbles)
    c=sample2['review_score'],  # c = color: review score 1–5
    cmap='RdYlGn',  # Red (bad) → Yellow → Green (good)
    alpha=0.6,  # Semi-transparent bubbles
    edgecolors='white',  # White outline for visual separation
    linewidth=0.5,
)
# Add a colorbar to decode the color scale
plt.colorbar(scatter, ax=axes[1], label='Review Score')

axes[1].set_xlabel('Price (BRL)')
axes[1].set_ylabel('Freight (BRL)')
axes[1].set_title(
    'Price vs Freight\n(bubble size = delivery days, color = review)',
    fontsize=12,
    fontweight='bold',
)
```



```
plt.tight_layout()
plt.savefig('olist_multivariate.png', dpi=150, bbox_inches='tight')
plt.show()
```





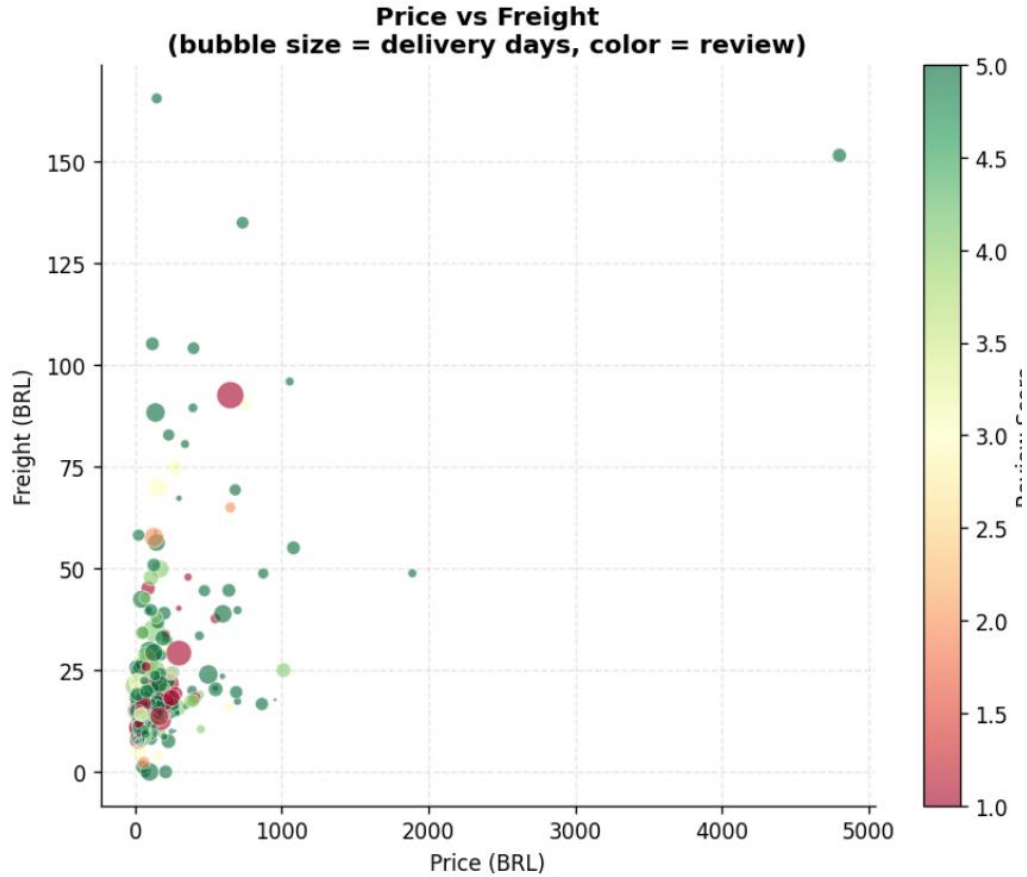


Plot 1 — Correlation Matrix

- **Price ↔ Freight (0.41)** — the strongest relationship here: heavier/bulkier items cost more to ship and tend to be priced higher, a moderate and expected link
- **Review ↔ Price (0.00)** — essentially zero, confirming what we saw earlier: how much you pay has no bearing on how satisfied you are
- **Review ↔ Freight (-0.03)** — also near zero, shipping cost alone doesn't drive satisfaction either
- **Delivery days ↔ Review (-0.30)** — the most actionable insight: longer delivery → lower review score, a weak-to-moderate negative relationship suggesting delivery speed genuinely affects satisfaction
- **Delivery days ↔ Freight (0.22)** — orders that take longer also tend to cost more to ship, possibly because they are going to remote or far destinations

Plot 2 — Price vs Freight Bubble Chart

- **Most orders cluster bottom-left** — the vast majority are low price (0–500 BRL) and low freight (0–50 BRL), consistent with the earlier scatter plot
- **Green dominates the cluster** — most orders in the dense region have review scores of 4–5, suggesting that typical, affordable orders generally satisfy customers
- **Red/pink bubbles appear in the mid-freight range** — unhappy customers (score 1–2) tend to cluster around moderate freight costs (25–75 BRL), possibly reflecting unmet delivery expectations, like a big red bubble in the middle which means long delivery period
- **Large bubbles (long delivery) are mixed in color** — some long-delivery orders get good reviews, some get bad ones, meaning delivery time alone doesn't fully determine satisfaction
- **High-price outliers (3000–5000 BRL) are green** — expensive orders in this sample appear to have satisfied customers, echoing the near-zero price-review correlation



Key takeaway:

> Delivery speed is the clearest lever for improving customer satisfaction — not price, not freight cost. Faster delivery = better reviews, and that signal is consistent across both plots.

7. Dashboard Design & Implementation

📊 What is a Dashboard?

> *"A dashboard is a visual display of the most important information needed to achieve one or more objectives, consolidated and arranged on a single screen so the information can be monitored at a glance."*
> — Stephen Few, *Information Dashboard Design*

A dashboard translates raw data into **actionable insights** for a specific audience. Unlike exploratory analysis (which is for the analyst), a dashboard is for **decision-makers who don't have time to run queries**.

✅ Good Practices

Principle	Implementation
5-Second Rule	The key message should be readable in 5 seconds. Test: show to someone for 5 seconds, ask "what's the main message?"
Limit to 5–7 panels	More panels = cognitive overload. Every panel must earn its place
KPI cards at the top	Always lead with 3–4 key metrics. Viewers read top-to-bottom
Consistent color convention	Pick one color for "good", one for "bad", use them throughout
Clear labels and units	Never make the viewer guess what the y-axis measures
Pre-calculated metrics	Show R$ 137 avg, not raw data the viewer must calculate

Understanding Matplotlib's Grid Layout for Dashboards

Creating a Custom Grid with `GridSpec`

`plt.subplots(rows, cols)` gives equal-sized panels. For dashboards, we need **panels of different sizes**.
That's where `GridSpec` comes in:

```
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(18, 14))
gs = gridspec.GridSpec(
    nrows=3,    # 3 rows
    ncols=3,    # 3 columns
    figure=fig,
    hspace=0.5,    # Vertical space between rows (fraction of avg axis height)
    wspace=0.4,    # Horizontal space between columns
    left=0.07,     # Left margin (fraction of figure width)
    right=0.97,    # Right margin
    top=0.88,      # Top margin (leaves room for title)
    bottom=0.07    # Bottom margin
)

# Add subplots at specific grid positions
ax_top_left   = fig.add_subplot(gs[0, 0])    # row 0, col 0
ax_top_right  = fig.add_subplot(gs[0, 1:3])  # row 0, cols 1-2 (wide panel!)
ax_full_width = fig.add_subplot(gs[1, :])    # row 1, all 3 cols
```

Our Dashboard Design Brief

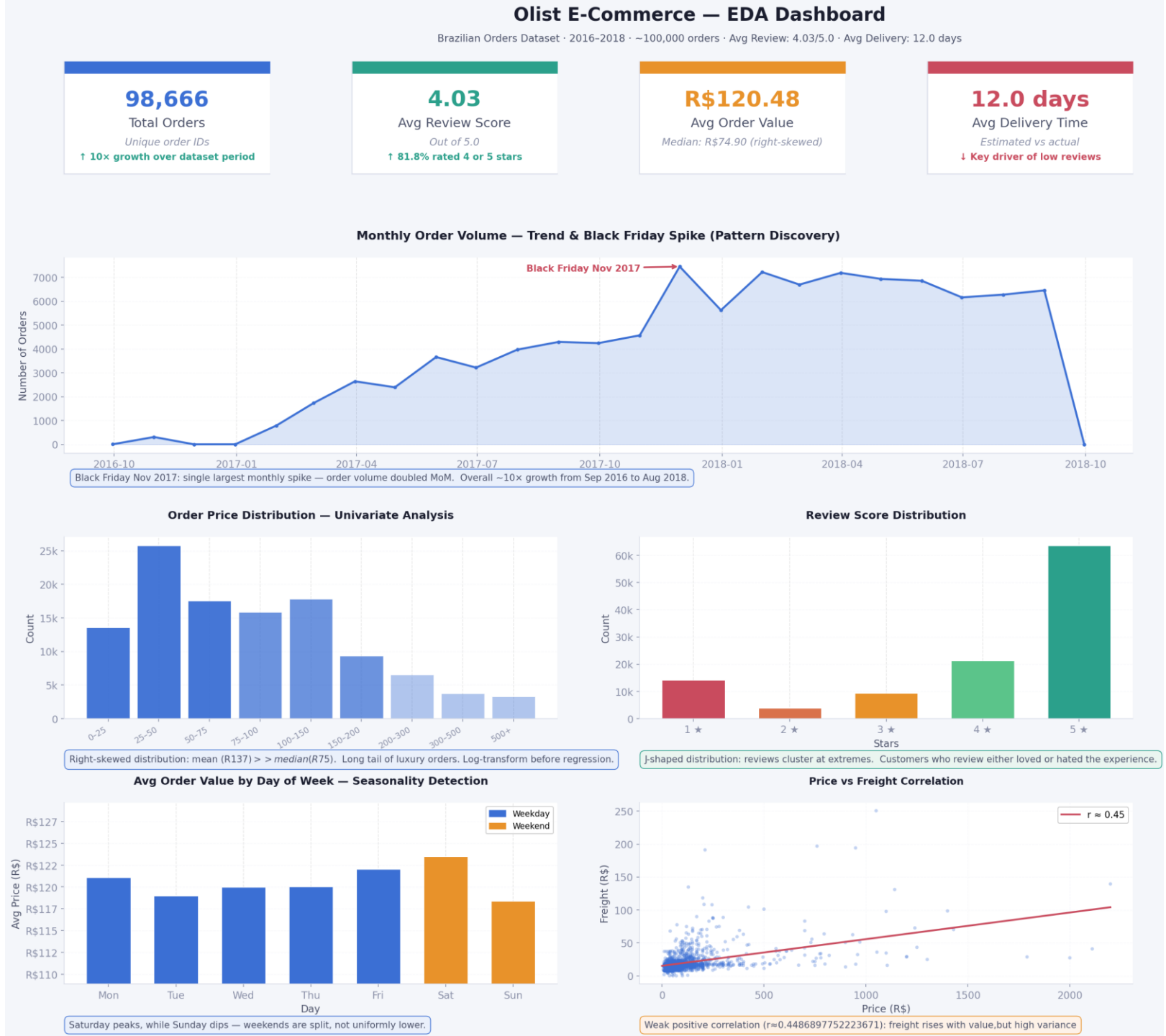
Story: *"What drives customer satisfaction on Olist?"*

Audience: Operations & Customer Experience Manager

Panel	Content	Plot Type
KPI Row (top, 4 cards)	Avg Review Score | Avg Delivery Days | Total Orders | Revenue	Metric Cards
Main Chart (wide)	Monthly order volume with trend & Black Friday annotation	Line Chart
Middle Left	Review score distribution	Bar Chart
Middle Right	Delivery days vs review score (scatter)	Scatter Plot
Bottom Left	Price distribution histogram	Histogram
Bottom Right	Orders by day of week	Bar Chart

Color convention:

- 🔴 **Red** → Low ratings, bad delivery performance
- 🟢 **Green** → High ratings, good performance
- 🔵 **Blue** → Neutral information (orders, prices)



The complete dashboard generation code is in the lab notebook.